Lab 24 — Data at Rest: LUKS2 Disk Encryption

> **SY0-701 Objective 3.3** — Compare and contrast concepts and strategies to protect data
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~15 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Encrypt a volume with LUKS2, prove its contents cannot be read from the raw disk image while an unencrypted disk leaks the same data, then rotate the passphrase, add a keyfile and back up the header.

You'll build: A LUKS2 (AES-XTS, Argon2id) encrypted volume on a loop device with two key slots, a header backup, and a plaintext-versus-ciphertext comparison.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; no earlier lab needed; tools installed in step 1. Passphrases in this lab are placeholders — never reuse them.

Background

- **Data at rest** is stored data — disks, tapes, backups and object storage. **Full-disk encryption** (LUKS, BitLocker, FileVault) protects it if the device is lost or stolen; **file-level** and **database (TDE / column)** encryption protect narrower scopes.
- LUKS encrypts the data with a random **master key**. Each **key slot** stores that master key wrapped by one passphrase or keyfile, so credentials can be rotated without re-encrypting the data.
- **Argon2id** is a memory-hard key derivation function that makes brute-forcing a passphrase expensive. AES-XTS with a 512-bit key means AES-256 in XTS mode.
- The **header** holds the key slots: lose or corrupt it and the data is gone for good, so it is backed up and stored securely — key management is the real control.
- Disk encryption protects **powered-off** devices. Once unlocked and mounted, the data is plaintext to anyone with access to the running system.

Steps

1. Install cryptsetup

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y cryptsetup
modprobe dm_crypt 2>/dev/null || true
cryptsetup --version
```

2. Create a virtual disk on a loop device

A 64 MB file attached as a block device stands in for a laptop disk.

```
mkdir -p /root/lab24 && cd /root/lab24
dd if=/dev/zero of=vault.img bs=1M count=64 status=none
LOOP=$(losetup -f --show vault.img)
```

```
echo "$LOOP" > loopdev
echo "Virtual disk attached as $LOOP"
```

3. Format the device with LUKS2 and inspect the header

The passphrase is held in a root-only file so the commands are non-interactive. Argon2id memory is reduced to 64 MB for the playground (production defaults are much higher).

```
cd /root/lab24 && LOOP=$(cat loopdev)
printf '%s' 'Lab24-Vault-Pass-One' > pass1 && chmod 600 pass1
cryptsetup luksFormat -q --type luks2 --cipher aes-xts-plain64 --key-size 512 --hash
sha256 \
  --pbkdf argon2id --pbkdf-memory 65536 "$LOOP" pass1
cryptsetup luksDump "$LOOP" | grep -E 'Version|Cipher:|Key:|PBKDF|Memory|^  [0-9]+: luks2'
| head -12
```

4. Open the volume, create a filesystem and store data

Opening maps the decrypted view to `/dev/mapper/vault`; everything written there is encrypted on the disk.

```
cd /root/lab24 && LOOP=$(cat loopdev)
cryptsetup open --key-file pass1 "$LOOP" vault
mkfs.ext4 -q /dev/mapper/vault
mkdir -p /mnt/vault && mount /dev/mapper/vault /mnt/vault
echo "CONFIDENTIAL: payroll export 2024 - synthetic" > /mnt/vault/payroll.txt
ls -l /mnt/vault && lsblk "$LOOP"
```

5. Prove the plaintext is absent from the raw image

Lock the volume, then search both the LUKS image and an unencrypted disk holding the same file.

```
cd /root/lab24
umount /mnt/vault && cryptsetup close vault
dd if=/dev/zero of=plain.img bs=1M count=16 status=none && mkfs.ext4 -q -F plain.img
mkdir -p /mnt/plain && mount -o loop plain.img /mnt/plain
echo "CONFIDENTIAL: payroll export 2024 - synthetic" > /mnt/plain/payroll.txt
sync && umount /mnt/plain
echo "Unencrypted disk image: $(grep -a -c 'payroll export' plain.img) match(es)"
echo "LUKS-encrypted image  : $(grep -a -c 'payroll export' vault.img) match(es)"
```

```
Unencrypted disk image: 1 match(es)
LUKS-encrypted image  : 0 match(es)
```

6. Rotate the passphrase

`luksChangeKey` rewraps the master key with a new passphrase; the data is not re-encrypted and the old passphrase stops working.

```
cd /root/lab24 && LOOP=$(cat loopdev)
printf '%s' 'Lab24-Vault-Pass-Two' > pass2 && chmod 600 pass2
cryptsetup luksChangeKey "$LOOP" --key-file pass1 --pbkdf-memory 65536 pass2
cryptsetup open --test-passphrase --key-file pass1 "$LOOP" 2>/dev/null && echo "old
```

```
passphrase still works" || echo "old passphrase rejected"
cryptsetup open --test-passphrase --key-file pass2 "$LOOP" && echo "new passphrase
accepted"
```

7. Add a keyfile slot and back up the header

A random keyfile in a second slot allows unattended unlock (in production it would be sealed by a TPM or held in a KMS). The header backup is your recovery copy of the key slots.

```
cd /root/lab24 && LOOP=$(cat loopdev)
dd if=/dev/urandom of=luks.key bs=64 count=1 status=none && chmod 400 luks.key
cryptsetup luksAddKey "$LOOP" --key-file pass2 --pbkdf-memory 65536 luks.key
cryptsetup luksDump "$LOOP" | grep -E '^  [0-9]+: luks2'
[ -f vault-header.bak ] || cryptsetup luksHeaderBackup "$LOOP" --header-backup-file vault-
header.bak
ls -l vault-header.bak
cryptsetup open --key-file luks.key "$LOOP" vault && mount /dev/mapper/vault /mnt/vault
cat /mnt/vault/payroll.txt
umount /mnt/vault && cryptsetup close vault
```

Test it

```
cd /root/lab24 && LOOP=$(cat loopdev)
grep -a -c 'payroll export' vault.img
cryptsetup open --test-passphrase --key-file luks.key "$LOOP" && echo "keyfile unlocks"
cryptsetup open --test-passphrase --key-file pass1 "$LOOP" 2>/dev/null || echo "old
passphrase rejected"
```

Expected: `0` (no plaintext on the encrypted disk), `keyfile unlocks`, and `old passphrase rejected`. `luksDump` lists two key slots (0 and 1).

Troubleshooting

Symptom	Fix
`Cannot use device ... which is in use`	The volume is still open; run `umount /mnt/vault; cryptsetup close vault`.
`losetup: cannot find an unused loop device`	Run `modprobe loop`, or reuse the existing device shown by `losetup -j /root/lab24/vault.img`.
`luksFormat` is killed or very slow	Lower Argon2id memory further, for example `--pbkdf-memory 32768`.
`Requested header backup file ... already exists`	A previous run created it; keep it, the step skips re-creating it.

Challenge

Simulate header loss: copy `vault.img` to `broken.img`, overwrite the first 16 MB of the copy with zeros using `dd conv=notrunc`, show that it can no longer be opened, then restore the header from `vault-header.bak` with `cryptsetup luksHeaderRestore` and open it again. Explain why the header backup must be protected as carefully as the passphrase.

Reflection

A stolen, powered-off laptop and a laptop infected with malware while unlocked are both "data at rest" problems. Which one does full-disk encryption solve, and which control from Objective 3.3 addresses the other?

Exam tips

- Data at rest = FDE / file / database encryption; in transit = TLS, IPSec, VPN; in use = secure enclaves or confidential computing.
- FDE protects lost or stolen devices; it does nothing once the system is booted and unlocked.
- Key slots let you rotate or revoke credentials without re-encrypting; destroying all keys is crypto-erase (crypto-shredding).
- TPM stores keys tied to the platform; an HSM or KMS manages keys centrally for servers and cloud.

References

- cryptsetup / LUKS2 project — https://gitlab.com/cryptsetup/cryptsetup
- NIST SP 800-111 Guide to Storage Encryption Technologies for End User Devices — https://csrc.nist.gov/pubs/sp/800/111/final
- NIST SP 800-57 Part 1 Recommendation for Key Management — https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final
- RFC 9106 Argon2 — https://www.rfc-editor.org/rfc/rfc9106
